HelixCare AI Hospital – Implementation Plan

Overview

HelixCare AI Hospital is a proof-of-concept autonomous digital hospital composed of multiple specialized agents (microservices) that communicate via the NEXUS-A2A protocol. Each hospital department (triage, diagnostics, documentation, etc.) is represented by a FastAPI-based autonomous agent. These agents exchange tasks using JSON-RPC 2.0 over HTTPS and publish real-time status events (via SSE/WebSockets) for streaming updates[1][2]. Agents discover each other's capabilities through *agent cards* (JSON descriptors served at a standard `.well-known/agent-card.json` endpoint) and authenticate via **OpenID Connect (OIDC) JWT bearer tokens**, augmented by **mutual TLS** for transport security. This design aligns with ISO/IEC/IEEE 29148:2018 principles by explicitly tracing requirements to functionalities (the provided scenario matrices map use-case **Requirement IDs** to agent behaviors and test cases[3][4]).

In summary, HelixCare's architecture is an **agent mesh** where each agent performs a specific role (e.g. triage orchestration, consent verification) yet interoperates through a common protocol. This avoids a monolith by composing smaller services with standardized task contracts and events[5][6]. Below we outline the project structure, agents to implement, security setup (OIDC + mTLS), integration with the Nexus-A2A framework, and testing strategy.

Project Structure

The HelixCare POC repository will be organized for clarity and alignment with the Nexus-A2A reference implementation. A proposed directory layout:

```
helixcare-ai-hospital/
├── agents/
│   ├── triage-agent/             # ED Triage Agent (FastAPI service)
│   │   ├── app/
│   │   │   ├── main.py           # FastAPI application for triage logic
│   │   │   └── agent_card.json   # Agent's self-description (capabilities, methods, auth)
│   │   └── Dockerfile
│   ├── diagnosis-agent/
│   │   ├── app/
│   │   │   ├── main.py           # Diagnosis support agent logic (LLM calls for risk assess)
│   │   │   └── agent_card.json
│   │   └── Dockerfile
│   ├── openhie-mediator/
│   │   ├── app/
│   │   │   ├── main.py           # FHIR data retrieval agent logic
│   │   │   └── agent_card.json
│   │   └── Dockerfile
│   ├── transcriber-agent/
│   │   ├── app/
│   │   │   ├── main.py           # Telemed transcription orchestrator logic
│   │   │   └── agent_card.json
│   │   └── Dockerfile
│   ├── summariser-agent/
│   │   ├── app/
│   │   │   ├── main.py           # LLM-based SOAP note summarization logic
│   │   │   └── agent_card.json
│   │   └── Dockerfile
│   ├── ehr-writer-agent/
│   │   ├── app/
│   │   │   ├── main.py           # EHR note persistence logic (writes to SQLite DB)
│   │   │   └── agent_card.json
│   │   └── Dockerfile
│   ├── insurer-agent/
│   │   ├── app/
│   │   │   ├── main.py           # Orchestrates consent verification workflow (insurer perspective)
│   │   │   └── agent_card.json
│   │   └── Dockerfile
│   ├── provider-agent/
│   │   ├── app/
│   │   │   ├── main.py           # Provides records if consent valid (verifies consent text)
│   │   │   └── agent_card.json
│   │   └── Dockerfile
│   ├── consent-analyser/
│   │   ├── app/
│   │   │   ├── main.py           # AI agent that parses/analyzes consent documents via LLM
│   │   │   └── agent_card.json
│   │   └── Dockerfile
│   ├── hitl-ui/
│   │   ├── app/
│   │   │   ├── main.py           # Human-in-the-loop approval stub agent
│   │   │   └── agent_card.json
│   │   └── Dockerfile
│   ├── hospital-reporter/
│   │   ├── app/
```

```
│   │   │   ├── main.py          # Agent providing hospital case statistics
│   │   │   └── agent_card.json
│   │   └── Dockerfile
│   ├── osint-agent/
│   │   ├── app/
│   │   │   ├── main.py          # Agent providing OSINT (news headlines) data
│   │   │   └── agent_card.json
│   │   └── Dockerfile
│   └── central-surveillance/
│       ├── app/
│       │   ├── main.py          # Public health orchestration agent (aggregates multiple inputs)
│       │   └── agent_card.json
│       └── Dockerfile
├── shared/
│   └── nexus_common/            # (Optional) Reused library code from nexus-a2a (JSON-RPC handling, auth, SSE,
etc.)
├── docker-compose.yml           # Orchestration of all agent services + support services
├── .env                         # Environment variables (JWT secret, OIDC config, API keys, etc.)
├── tests/
│   ├── scenario_matrices/       # JSON scenario files (e.g. nexus_ed_triage_matrix.json, etc.)
│   └── test_harness.py          # (Optional) Script or pytest to drive scenarios using the matrices
└── docs/
    └── README.md                # Documentation of the system and usage
```

Notes: - Each agent has its own subfolder with a FastAPI app and an `agent_card.json` describing its name, purpose, URL, supported methods, and auth schemes. We will reuse the expertly defined agent cards from the HelixCare artifacts (ensuring they align with each agent's role). For example, the **TriageAgent** card lists its methods and that it uses bearer-token auth[7][8]. - The `shared/nexus_common` package will contain common protocol code (possibly pulled in from the `sync-ai-health/nexus-a2a-protocol` repo). This includes JSON-RPC parsing, JWT verification, SSE event bus, HTTP client helpers, etc., which all agents will import to avoid duplicating code. By using this shared library (or installing `nexus-a2a-protocol` as a dependency), we ensure consistency with the reference implementation (e.g. strict JSON-RPC compliance and error handling as in `jsonrpc.py`, JWT scope checking in `auth.py`, etc.)[9]. - The **docker-compose.yml** in the root defines all agents and supporting services (Redis, MQTT broker, FHIR server, etc.) for one-step deployment. We encapsulate environment configuration (secrets, URLs) in a central `.env` file for convenience.

Agents and Their Responsibilities

HelixCare consists of multiple autonomous agents, each mapped to a functional role or department. The agents collaborate to fulfill complex workflows (triage, documentation, etc.). Below is the list of agents to implement and their purposes:

- **TriageAgent (ED Triage Department)** – Orchestrates the emergency department triage workflow[10]. This agent receives triage requests (patient demographics, chief complaint, etc.) and coordinates a risk assessment:
- It delegates clinical assessment to the DiagnosisAgent and integrates the results. The triage agent's `tasks/sendSubscribe` method triggers this flow asynchronously[11][12]: it accepts a triage task, publishes an "accepted" event, then calls the remote **DiagnosisAgent** via JSON-RPC (`diagnosis/assess` method) with the patient info.
- After receiving the diagnosis results (e.g. a triage priority and rationale), TriageAgent emits a final result event and could determine the triage outcome (e.g. category of urgency). It essentially acts as an orchestrator and decision-maker for ED intake[10], ensuring the patient is classified with appropriate priority.

- **DiagnosisAgent (Diagnostics Dept)** – Performs AI-driven clinical assessment of a patient's condition[13]. It takes inputs like chief complaint and patient context, and returns a triage recommendation:

- Upon receiving a `diagnosis/assess` JSON-RPC task from TriageAgent, it fetches the patient's clinical context from the EHR via the OpenHIEMediator (see below) and then uses an LLM to analyze the complaint in context[14][15]. For example, it might identify keywords (e.g. "chest pain") to assign a high acuity ("EMERGENCY") and generate a brief rationale via an LLM[16].
- It returns a result containing the suggested **triage priority**, an explanation/rationale, and the fetched patient context[15]. This agent encapsulates diagnostic reasoning as a service. (In a real hospital, similar logic could extend to radiology or lab diagnostics; here it's focused on triage support.)

- **OpenHIEMediatorAgent (Interoperability – FHIR Gateway)** – Acts as a mediator to retrieve patient records (FHIR resources) needed for care:

- This agent is invoked by the DiagnosisAgent to fetch data like patient demographics or allergies from a FHIR server. The agent exposes a method `fhir/get` to retrieve a Patient resource and related records[17][18]. In the POC, it queries a HAPI FHIR server (running as a support service) for a given patient ID. The result (patient JSON and any allergies) is passed back to the caller[19][20].
- OpenHIEMediator ensures that agents can easily integrate with standards-based health information exchanges. In this design, it is used in the triage workflow to supply context to the AI (simulating how an agent could pull info from the hospital EMR).

- **TranscriberAgent (Telemedicine Scribe Orchestrator)** – Coordinates real-time clinical documentation during a telemedicine encounter. It represents a scribe that turns conversation into a structured note:

- It accepts a transcription task (containing, for example, a raw transcript of a doctor-patient conversation) and runs a two-step process asynchronously[21][22]. First, it calls the SummariserAgent to generate a summarized SOAP note from the transcript[23]. Then it calls the EHRWriterAgent to save that note to the record[22].

- The TranscriberAgent's `tasks/sendSubscribe` method orchestrates these steps, emitting status updates like "summarising" and "saving_ehr" in the event stream[23][24]. Once the note is saved, a final result (with confirmation and any record ID) is published. This agent effectively automates documentation: it ensures that after a telemedicine session, a formatted note is created and stored with minimal human effort.

- **SummariserAgent (Clinical Documentation)** – An AI agent that transforms raw text into a structured medical note. It uses an LLM to produce SOAP (Subjective, Objective, Assessment, Plan) note content:

- It offers a `note/summarise` RPC method. The agent takes as input a transcript or conversation text and returns a Markdown-formatted note[25]. Internally, it prompts an LLM (e.g., an OpenAI model via API) with a system instruction to act as a medical scribe and the transcript as user input[25]. The output is a concise note in markdown, which might include sections for subjective info, objective findings, assessment, and plan.
- By separating this function, the SummariserAgent can be improved or replaced independently (for example, pointing it to different LLM models or adding more advanced parsing). Its agent card indicates it is capable of the "SOAP Note Summariser" skill[26].

- **EHRWriterAgent (Electronic Health Record Integration)** – This agent persists clinical notes or data into a database, simulating integration with an EHR system:

- It provides methods like `ehr/save` (to store a note) and `ehr/getLatestNote` (to retrieve the latest note, for verification)[27][28]. In the POC, EHRWriterAgent uses a simple SQLite database (with a table for notes) to persist data[27]. When the TranscriberAgent invokes `ehr/save` with a patient identifier and note content, this agent writes a record and returns a success flag[27].
- The use of a real database (even a lightweight one) demonstrates stateful behavior and the ability to later query or audit inserted records. In a real deployment, this agent could interface with a hospital's actual EHR API or FHIR server to create clinical notes.

- **InsurerAgent (Consent Verification Orchestrator)** – Orchestrates a workflow to handle a request for patient records that requires checking patient consent. You can think of this as the "requesting party" (e.g., an insurer or external entity asking for records):

- When a task comes in (e.g. "Please provide patient X's discharge summary if consent allows"), the InsurerAgent coordinates the steps. Its `tasks/sendSubscribe` method first asks the ProviderAgent for the records[29]. Notably, the InsurerAgent doesn't directly have the data – it relies on the provider/hospital side agent to decide if releasing the data is authorized.
- The InsurerAgent then enforces a human-in-the-loop (HITL) gate if required. In the HelixCare POC, an environment flag can indicate that any authorized data release must be approved by a human reviewer. The InsurerAgent will, after getting a positive authorization from ProviderAgent, call the HITL-UI Agent to simulate a manual approval step[30]. This step is marked with an "input-required" state event (awaiting approval) and then an approval decision is obtained[30].
- Finally, the InsurerAgent returns the outcome: if consent was not present, it might return an error or denial reason; if approved, it returns the requested record data (e.g., a summary) along with evidence of approval. This agent ensures that **policy checks** (consent and human approval) wrap every cross-organization data request.

- **ProviderAgent (Data Provider & Consent Enforcer)** – Represents the hospital's side of record release. It is responsible for supplying the requested record *if and only if* proper consent exists:

- The ProviderAgent exposes `records/provide`. When invoked by the InsurerAgent, it will verify consent before actually returning any data[31][32]. The input includes a `consent_text` (the content of the patient's consent form or statement). ProviderAgent delegates the analysis of that consent text to the ConsentAnalyserAgent – it calls `consent/check` and gets a result indicating allowed/denied[33].
- If the consent analyser says **not allowed**, ProviderAgent immediately returns `{"authorized": false, "reason": "..."}`[34], declining the request with a reason (which could be "Consent does not cover this data" or similar). If allowed, it proceeds to fetch/prepare the requested record. In our POC, we simulate with a static example record (e.g., a discharge summary text) and return it with `{"authorized": true, "record": {...}}`[32]. This agent thus enforces patient consent at the point of data release – a critical governance step.

- **ConsentAnalyserAgent (AI Consent Checker)** – An AI agent that interprets the text of a consent form or authorization to decide if a particular data request is permitted:

- It implements the `consent/check` method. On input of a `consent_text`, it uses natural language processing (in this case a GPT-style model) to determine if the consent text grants permission for the requested action[35]. The logic in the POC is simplified: it prompts an LLM with "You are a consent checker. Reply ALLOW or DENY with reason" and then checks if the word "deny" is present in the model's answer[35]. If "deny" is not found, it assumes consent is granted (`allowed: true`), otherwise not. It returns a JSON with `allowed` flag and a short rationale (the AI's response as the "reason").
- Even though this is a simplistic AI reasoning, it demonstrates how an autonomous agent could dynamically interpret legal or textual documents. In a more advanced system, this could be replaced with a rules engine or a more robust NLP model, but the pattern remains: a separate agent dedicated to consent policy interpretation.

- **HITL-UI Agent (Human-in-the-Loop Approval)** – A stub agent to simulate a human approval interface. It stands for a user interface where a compliance officer or clinician could manually approve or reject a pending action:

- It provides a `hitl/approve` method. In the POC implementation, this agent simply auto-approves every request (since we can't truly pause for human input in an automated test)[36]. The method returns an approval decision (always `"approved": true` along with any provided comment) as if a human clicked "Approve" on a GUI[36].
- The presence of this agent in the architecture highlights that certain high-risk tasks can require human sign-off. In a real system, this agent would interface with an actual UI (e.g., a web dashboard where a human sees the request details and chooses to

approve or reject). For the POC, we include it to complete the workflow loop logically, and it can later be extended into a real UI component.

- **HospitalReporterAgent (Public Health – Hospital Data Source)** – Provides aggregated hospital statistics for public health surveillance. This agent simulates generation of a weekly report of case counts:

- Method: `surveillance/report`. When asked for data about a certain disease (*pathogen*) and region, it returns a JSON with fake statistics (e.g., number of cases and deaths in the past week)[37][38]. In our POC, it picks random numbers to simulate variability[38]. For example, a request `{"pathogen": "cholera", "region": "Gauteng"}` might produce `{"hospital": "Mock General Hospital", "region": "Gauteng", "pathogen": "cholera", "week": "2025-W03", "cases": 42, "deaths": 1, ...}`.
- This agent stands in for hospital internal reporting systems (like lab results, admission counts, etc.) that could feed into a broader surveillance network. It's lightweight and stateless (generating data on the fly).

- **OSINTAgent (Public Health – Open Source Intel)** – Gathers open-source intelligence (news headlines) about disease outbreaks. This agent responds with recent news related to a pathogen of interest:

- Method: `osint/headlines`. Input: a pathogen name. It returns a set of headline snippets relevant to that disease[39][40]. In our POC, we use a hardcoded dictionary of example headlines for a few diseases (cholera, measles, TB, influenza)[39][40]. For pathogens not in the dictionary, it returns a default "No OSINT data" message.
- The OSINTAgent mimics pulling data from news APIs or web scraping, but for POC we avoid external calls and use static content. It provides external context that, combined with hospital data, can help determine if there is an outbreak or public health issue emerging.

- **CentralSurveillanceAgent (Public Health – Central Authority)** – Orchestrator that takes inputs from multiple sources (hospital reports, OSINT) to assess public health status and generate alerts. This agent represents a public health center receiving data and using AI to synthesize an actionable alert:

- It offers a `tasks/sendSubscribe` (or similar) endpoint (we can name it `surveillance/analyze` for clarity) that expects parameters like a pathogen and region of concern. When triggered, it performs a **fan-out** to gather data: it will request the hospital stats and OSINT in parallel, preferring **MQTT** pub/sub channels for speed[41][42].
    - The agent attempts to publish a request to an MQTT topic (e.g., `nexus/surveillance/report/req`) and waits briefly for a response on a corresponding topic[41][42]. If the MQTT broker is available and the other agent is listening, this yields the fastest response. If not (e.g., broker down or agent not subscribed), it falls back to regular HTTP JSON-RPC calls to HospitalReporter and OSINT agents[42][43]. This dual-transport strategy demonstrates **multi-transport** support (requirement IR-2), which the Nexus protocol supports.
- Once it obtains the hospital data and OSINT data (each either via MQTT or HTTP), the Central agent then synthesizes an **outbreak alert** using an LLM[44][45]. It constructs a prompt containing the collected data and instructs the LLM (e.g., "Given the hospital report and news, produce a concise outbreak alert with an alert level and recommendations")[44][45]. The LLM's output (expected to be JSON) is parsed or wrapped into a structured alert object with fields like `alert_level` ("green/yellow/orange/red"), `summary`, and `recommended_actions`.
- The final alert, along with the input data and an indicator of which transport was used (MQTT or HTTP), is returned to the requester[46]. Throughout this process, the CentralSurveillanceAgent streams status events: e.g., "fan-out" started, then "synthesis" started, then "completed"[47][44]. This agent showcases a complex multi-agent, multi-transport integration with an AI-driven decision at the end – effectively simulating how a regional public health center might automatically detect and alert on outbreaks.

Each agent's implementation will follow a consistent pattern: FastAPI app with a `/rpc` POST endpoint to handle JSON-RPC tasks, an `/events/{task_id}` SSE endpoint for streaming events, a `.well-known/agent-card.json` for discovery, and a `/health` endpoint for health checks. The *agent card* of each agent defines its name, description, supported methods, and required auth. For instance, the **SummariserAgent's** card describes it as *"Summarises clinical transcripts into SOAP notes using LLM"* and lists `note/summarise` as its method[26]; the **ConsentAnalyser's** card would list the `consent/check` method and indicate it uses bearer auth, etc. We will use these descriptions to configure each agent's metadata.

Inter-Agent Communication & Workflow

All agents communicate via the **NEXUS-A2A protocol**, which is essentially JSON-RPC 2.0 carried over HTTP(S) with some conventions for asynchronous tasks and streaming results:

- **JSON-RPC Task Exchange:** Agents send tasks to each other by making HTTP POST requests to the target agent's `/rpc` endpoint with a JSON-RPC formatted payload. For example, the TriageAgent calls the DiagnosisAgent by posting `{"jsonrpc": "2.0", "id": "...", "method": "diagnosis/assess", "params": {...}}` to `http://diagnosis-agent:8022/rpc`[48]. Each agent's `main.py` includes a generic dispatcher that routes incoming RPC calls to the appropriate handler coroutine (if the method is recognized)[1][49]. Responses are returned as JSON-RPC results or errors.

- **Asynchronous Task Handling:** Some tasks are long-running or multi-step (e.g. triage orchestration, surveillance analysis). For these, the pattern used is **fire-and-stream**: the initiating agent (client) calls a `sendSubscribe` method on the orchestrator agent, which immediately returns a `task_id` (and possibly a trace id) while the work continues in background[11][50]. The orchestrator agent will then emit events on a pub/sub bus as the task progresses. For instance, TriageAgent's `_send_subscribe` handler spawns a background job for calling DiagnosisAgent, and returns quickly with `{"task_id": "...", "trace_id": "..."}`[50]. This allows the calling client (or a UI) to start listening for events corresponding to that task.

- **Event Streaming (SSE/WebSockets):** Each agent includes an `/events/{task_id}` endpoint (Server-Sent Events) and a `/ws/{task_id}` endpoint for WebSocket, enabling clients or other agents to subscribe to task updates[2][51]. In the POC, we primarily use SSE for simplicity. When an orchestrator publishes an event (like status or final result) to the TaskEventBus, it is broadcast via Redis pub/sub to any listeners. For example, the TranscriberAgent will publish events "accepted", "working (summarising)", "working (saving_ehr)", then "final"[23][24]. A client monitoring `/events/{task_id}` on TranscriberAgent will receive these as SSE data in real time. This streaming fulfills requirement FR-7 (real-time workflow updates)[52][53].

- **Agent Discovery:** Agents need a way to find each other's endpoints and capabilities. HelixCare employs two discovery mechanisms:

- **Static configuration via environment** – In our docker-compose, each agent is given the URLs of the agents it needs to call (e.g., TranscriberAgent has `SUMMARY_RPC=http://summariser-agent:8032/rpc` in env[54], TriageAgent has `NEXUS_DIAGNOSIS_RPC=http://diagnosis-agent:8022/rpc`[55], etc.). This is a form of pre-configuration or service discovery by environment.
- **Agent Card lookup** – Each agent serves its `agent_card.json` at `/.well-known/agent-card.json`[56]. This JSON includes the agent's base URL and supported methods and auth schemes. In a dynamic scenario, an agent can query another agent's card (either directly via a known URL or via a registry service) to discover how to interact with it. Our test harness uses this (the scenario matrices include steps to GET the agent card as part of discovery verification)[57][58]. In a production setup, we could also run a simple directory service or have agents announce themselves to a registry, but for the POC, the `.well-known` endpoint and known hostnames suffice.
- In essence, every agent is independently deployable and publishes enough info (via its card) for others to interoperate with it[59].

- **Strict Interface Contracts:** The tasks exchanged are defined by method names like `diagnosis/assess`, `note/summarise`, etc., which act like API endpoints. Each expects certain params and returns certain data structures (as we detailed per agent). Using JSON-RPC ensures a standard envelope with error codes for issues like method not found, invalid params, etc. (e.g., our agents use a shared helper to parse requests and will return a `-32601 Method not found` error if an unknown method is called[60]). This strictness was part of the requirements (CR-1: JSON-RPC envelope compliance, NFR-3: proper error codes)[61][62]. We will adhere to the same by leveraging the Nexus common library which already implements these checks.

- **Traceability and Logging:** Each task is identified by a `task_id` (often a UUID) and agents also propagate a `trace_id` for logging correlation across services[63][64]. In debug logs, we log when a task is accepted and completed along with these IDs for end-to-end tracing. The TaskEventBus events also carry the task_id and optionally a trace or parent id to correlate sub-tasks (e.g., TriageAgent gave the call to DiagnosisAgent an ID of `task-xyz-diag` in the code)[48]. This helps in monitoring and is surfaced in the Command Centre UI.

Overall, the communication approach ensures that each agent can focus on its function but still participate in larger workflows by invoking others through a uniform protocol. The use-case scenario matrices provided act almost like integration tests for these workflows – for example, **ED Triage scenario** (FR-3) involves TriageAgent calling DiagnosisAgent and OpenHIEMediator in sequence[3][65], and **Public Health scenario** covers multi-transport fan-out (IR-2) and streaming (FR-7)[52][66]. Our implementation plan explicitly covers these interactions as described above.

Security: Authentication and Authorization (OIDC, JWT, mTLS)

Security is critical in a hospital context. HelixCare will implement multiple layers of security for agent-to-agent calls:

- **OIDC and JWT-Based Auth:** All agent RPC calls require a valid **bearer token** in the HTTP `Authorization` header. Each agent's card advertises that it uses `"authentication": {"schemes": ["bearer"]}`[67][68], meaning a JSON Web Token must accompany requests. In the POC, we will use a single JWT signing secret (shared via environment `NEXUS_JWT_SECRET`) to sign and verify tokens for simplicity – this is already integrated in the Nexus common `verify_jwt()` function each agent calls on incoming requests[69][70]. The token must carry an appropriate scope claim (e.g. `"scope": "nexus:invoke"`) which is checked server-side[71][70]. This satisfies **SPR-1 (Bearer token auth)** and **SPR-5 (scope-based authorization)**[72].

For a more robust setup, we will incorporate **OpenID Connect**: - Deploy a lightweight OIDC provider (for example, Keycloak or MITRE IdP) via docker-compose. This IdP will manage client credentials for each agent. Each agent can be given an OIDC client ID/secret, or we can use a single technical client for all internal services if appropriate. - Use **Client Credentials Flow**: An agent that needs to call another can obtain a JWT from the IdP's token endpoint, which will be signed by the IdP's private key and include scopes (e.g., the "nexus:invoke" scope indicating an internal service call). For the POC, we might pre-generate a long-lived token to avoid adding complexity of live token requests, but we will simulate the process. - Configure each agent's JWT verification to trust the IdP's signature. This can be done by either using a common secret (if using symmetrical signing like HS256 as in the POC) or better, by configuring `verify_jwt` with the IdP's public key or JWKS endpoint (for RS256 tokens). For now, our code uses HS256 with a shared secret for simplicity[71], but we can later extend `verify_jwt` to validate RS256 tokens using the OIDC discovery metadata (which includes JWKS URL). - This OIDC integration ensures that in a real deployment, credentials are centrally managed, tokens can be short-lived, and each agent doesn't have to share a static secret. It also aligns with healthcare standards (e.g., SMART-on-FHIR uses OIDC/JWT for service auth).

- **Mutual TLS (mTLS):** To protect data in transit and ensure we know which agent is calling, we will use mutual TLS for all HTTP communications between agents. In the POC environment, this means:
- Generate a local Certificate Authority (CA) and issue client certificates to each agent service (each agent container will have its own cert + key and truststore containing the CA). We can automate this with a script or use a tool like mkcert for development.
- Terminate TLS either directly in each FastAPI app (for instance, running Uvicorn with SSL and requiring client certificate verification) or via an Nginx/haproxy sidecar. A simple approach is to run an Nginx reverse proxy in front of each agent that is configured with `ssl_verify_client on;` and the CA, forwarding to the local FastAPI app. Another approach is a centralized service mesh or proxy that enforces mTLS. For the POC, adding per-service Nginx might complicate the setup, so we could

alternatively run all agents on an internal Docker network and have a single entrypoint handle TLS termination just for external access. However, since the requirement calls out mTLS, we'll demonstrate it at least between a couple of agents as a proof: for example, set up mTLS between the InsurerAgent and ProviderAgent communication.

- With mTLS, each agent will only accept connections from clients presenting a valid certificate signed by the trusted CA. This provides an extra authentication layer at the transport level, mitigating risks of impersonation or man-in-the-middle. It complements the JWT auth (one is transport-level, one is message-level). Agents will still use JWTs internally, but even if a token were stolen, an attacker without a valid cert couldn't open a connection to an agent.
- We will document how to run the compose with mTLS (likely by enabling an Nginx proxy container for each relevant agent, mounting the certs). This part might be optional to turn on in development (via an env flag), since it requires generating certificates.

- **Access Control and Scoping:** Each JWT includes scopes, and the `verify_jwt` function enforces that the token contains the required `"nexus:invoke"` scope for inter-agent calls[70]. We can expand on this by giving different agents different scopes if needed (e.g., maybe only certain agents can invoke certain methods). For now, a single scope for all internal calls is used (this matches the Nexus demo setups). The scenario matrices include tests for missing or bad tokens to ensure a 401 Unauthorized is returned (NFR-1 and SPR-1)[72]. Our implementation will pass those by using the provided auth middleware (which already returns 401 on missing token or invalid signature)[69].

- **Agent Identity (DID integration):** In future, the architecture anticipates decentralized identity (DID) usage (SPR-2) where each agent could have a DID and sign requests. The code already has hooks for DID signature verification (the `verify_did_signature()` calls guarded by a feature flag)[70]. For the POC, we will keep DID verification turned **off** (as indicated by `DID_VERIFY=false` in env) unless we have a simple way to generate DID keys. But the presence of this in the codebase means the design is forward-compatible with strong identity proofs. We'll mention this as a possible extension: e.g., issuing each agent a DID Document and using a VC (verifiable credential) to prove its role. That goes beyond our immediate scope.

To summarize security: **Every RPC call must be over HTTPS (with client cert) and include a valid JWT from a trusted authority.** If a call lacks the token or has wrong scope, it is rejected with HTTP 401[73]. If an agent tries to call a method it's not allowed or that doesn't exist, a JSON-RPC error is returned but the connection is still authenticated. By combining OIDC (identity and authZ) and mTLS (transport authentication), HelixCare ensures a zero-trust approach where even internal traffic is protected and authenticated at multiple layers. This approach aligns with modern healthcare security frameworks (as noted in the white paper, "Identity and Trust Layer" is a core part of the architecture[74]).

Deployment with Docker Compose

We will use **Docker Compose** to containerize and run the entire HelixCare POC stack locally. This makes it easy to launch all agents and supporting services with one command and ensures the environment variables and network links are correctly set up for inter-agent communication.

Supporting Infrastructure Services: - **Redis:** A Redis instance (latest alpine image) on port 6379 will be included. This is used by the Nexus `TaskEventBus` in each agent for publish/subscribe of events[75][76]. All agents will have `REDIS_URL=redis://redis:6379` in their environment and connect on startup. Redis enables cross-container event propagation so that an event published by, say, the DiagnosisAgent can be consumed by the Command Centre or any client subscribed to that agent's SSE. We mark Redis as a dependency for agents in compose, and expose it if needed for debugging (not strictly required to access from host). - **MQTT Broker:** We include an Eclipse Mosquitto broker service on port 1883 (and 9001 for WebSocket MQTT if needed)[77]. This allows the CentralSurveillanceAgent and others to use MQTT for pub/sub. The broker is configured (via a mosquitto.conf if needed) to allow anonymous local connections in POC. In compose, we add healthcheck to ensure broker is up before dependent services start[78]. HospitalReporter and OSINTAgent will not directly depend on MQTT (they use HTTP by default), but CentralSurveillanceAgent will depend_on mosquitto (with condition healthy) since it tries to use it[79]. - **HAPI FHIR Server:** For realistic FHIR data, we run the `hapiproject/hapi` image on port 8080[80]. This is a full FHIR R4 server. In POC we might preload it with a sample patient. OpenHIEMediatorAgent uses the `FHIR_BASE_URL` env to know where to fetch data (set to `http://hapi-fhir:8080/fhir` by default)[19]. The OpenHIEMediator container depends on HAPI (so that it doesn't try to fetch before server is up)[81]. This service lets the DiagnosisAgent retrieve patient info and demonstrate standards-based exchange. - **OpenID Connect Provider:** (Optional in initial POC) – If we include Keycloak, it will run on e.g. port 8081 with an initial realm setup. For simplicity, we might use a dev JWT approach first. But in final plan, we note that an IdP is part of deployment if this were production – perhaps not fully configured in this compose due to time, but the code is ready to integrate by adjusting `verify_jwt`. If we add it, we ensure all agents have the IdP's URL and credentials. Alternatively, we can omit it and stick to static JWT for now (given the tests in matrices assume a static `NEXUS_JWT_SECRET`). We'll mention it as commented service in compose.

- **Jaeger (Tracing):** The ED triage demo included a Jaeger all-in-one for distributed tracing[82]. We can optionally include Jaeger (exposing UI on 16686) and configure agents to send trace spans (the code may have integration for this, or we add an OpenTelemetry middleware). This is not strictly required, but it's a nice-to-have for debugging. Since our focus is core functionality, we might skip for now or leave commented.

Now, each **Agent Service** in the compose file: We will follow patterns from the existing demo compose files, merging them into one configuration. Key points for each:

- Set the build context to the repo root and Dockerfile path to each agent's Dockerfile, so we can build images for each agent. The Dockerfiles will likely be very similar (install dependencies, copy code, run Uvicorn on `$PORT` env).
- Assign each agent a container name and internal hostname (the service key doubles as hostname for inter-service communication).
- Expose the agent's port (we use distinct ports 8021–8053 as given by the demos, which avoids any conflict).
- Provide environment variables:
- `PORT`: the port FastAPI will listen on (inside container). (We use the same number as the external for clarity).

- `NEXUS_JWT_SECRET`: loaded from `.env` (a strong dev secret or base64 string).
- `NEXUS_REQUIRED_SCOPE`: set to `nexus:invoke` (consistent across all)[55].
- `REDIS_URL`: `redis://redis:6379` for event bus[81].
- `DID_VERIFY`: can be `false` (or not set) to disable DID signature checking in all agents (unless we test that feature).
- Any agent-specific config:
  - For TriageAgent: `NEXUS_DIAGNOSIS_RPC=http://diagnosis-agent:8022/rpc`[83] so it knows where to send the sub-task.
  - For DiagnosisAgent: `NEXUS_OPENHIE_RPC=http://openhie-mediator:8023/rpc`[84] to fetch FHIR data; also OpenAI API key env (`OPENAI_API_KEY`, `OPENAI_MODEL`) since it calls the LLM[85].
  - OpenHIEMediator: `FHIR_BASE_URL=http://hapi-fhir:8080/fhir`[86].
  - TranscriberAgent: `SUMMARY_RPC=http://summariser-agent:8032/rpc`, `EHR_RPC=http://ehr-writer-agent:8033/rpc`[54].
  - SummariserAgent: `OPENAI_API_KEY` and `OPENAI_MODEL` (model could default to gpt-3.5 or similar)[87].
  - EHRWriterAgent: `EHR_DB=/data/ehr.sqlite` (and we mount a volume for /data to persist the SQLite)[88].
  - InsurerAgent: `PROVIDER_RPC=http://provider-agent:8042/rpc`, `HITL_RPC=http://hitl-ui:8044/rpc`, and optionally `HITL_REQUIRED=true/false` to toggle the gate[89].
  - ProviderAgent: `CONSENT_RPC=http://consent-analyser:8043/rpc`[90].
  - ConsentAnalyserAgent: `OPENAI_API_KEY` (for its LLM usage)[91].
  - CentralSurveillanceAgent: `HOSPITAL_REPORTER_URL=http://hospital-reporter:8051`, `OSINT_AGENT_URL=http://osint-agent:8052`, `MQTT_BROKER=mosquitto`, `MQTT_PORT=1883`[79]. These allow it to reach the other two data sources and the broker.
  - The HospitalReporter and OSINT agents in our simple case don't need special env besides the common ones. We might include an `AGENT_NAME` env for logging clarity (some demos set `AGENT_NAME` similarly to the service name).
- All agents that use OpenAI API share the same `OPENAI_API_KEY` which we'll set in `.env` (developers will plug in their key for testing, or we stub out the llm calls if offline).

- Define service dependencies (depends_on):

- Many agents depend on `redis` being up (for SSE). We set `depends_on: redis` for all, though not strictly all need it before startup (they'll retry connecting), but for safety we do.
- Where one agent calls another at startup or quickly, we ensure ordering: e.g., TriageAgent depends_on DiagnosisAgent (which depends_on OpenHIEMediator)[92], so that by the time triage starts, diagnosis is ready, and openhie is ready for diagnosis. Similarly, Transcriber depends_on Summariser & EHRWriter[93], Insurer depends_on Provider & HITL (and provider depends_on consent)[94][95], Central depends_on HospitalReporter & OSINT & MQTT[96]. These mimic the sequences in demo files.
- The compose will ensure, for example, that the ProviderAgent starts only after the ConsentAnalyser (since Provider will call it)[97][98].

- The **Command Centre**: We include a **command-centre** service in compose as well. This uses the provided image/build from `shared/command-centre`. It aggregates the monitoring of all agents:

- We'll configure `AGENT_URLS` to include all agent base URLs (http://triage-agent:8021, http://diagnosis-agent:8022, ... etc.)[99]. Given we have many agents, we'll list them comma-separated. This service will poll each agent's `/health` endpoint and subscribe to their events to build a live dashboard[100][101].
- Expose command-centre on `8099`, so the user can access the web UI at `http://localhost:8099` to see a network graph and metrics. In compose, ensure it depends_on all agents (so it doesn't start before they register in Redis)[102].
- The command centre is optional for functionality, but highly useful for demonstration and debugging. It will visually confirm that agents are communicating as expected (e.g., it will show edges like triage-agent → diagnosis-agent dependency[103][104], and highlight any errors).

A fragment of the **docker-compose.yml** (for a couple of services) will look like this:

```
services:
  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]

  mosquitto:
    image: eclipse-mosquitto:2
    ports: ["1883:1883", "9001:9001"]
    healthcheck:
      test: ["CMD-SHELL", "mosquitto_sub -t '$$SYS/#' -C 1 -W 3 || exit 1"]
      interval: 5s
      retries: 3

  hapi-fhir:
    image: hapiproject/hapi:latest
    ports: ["8080:8080"]

  triage-agent:
    build:
      context: .
      dockerfile: agents/triage-agent/Dockerfile
    environment:
```

```
    - PORT=8021
    - NEXUS_JWT_SECRET=${NEXUS_JWT_SECRET}
    - NEXUS_REQUIRED_SCOPE=nexus:invoke
    - NEXUS_DIAGNOSIS_RPC=http://diagnosis-agent:8022/rpc
    - REDIS_URL=redis://redis:6379
    - DID_VERIFY=${DID_VERIFY:false}
  ports: ["8021:8021"]
  depends_on:
    diagnosis-agent:
      condition: service_started
    redis:
      condition: service_started

diagnosis-agent:
  build:
    context: .
    dockerfile: agents/diagnosis-agent/Dockerfile
  environment:
    - PORT=8022
    - NEXUS_JWT_SECRET=${NEXUS_JWT_SECRET}
    - NEXUS_REQUIRED_SCOPE=nexus:invoke
    - NEXUS_OPENHIE_RPC=http://openhie-mediator:8023/rpc
    - OPENAI_API_KEY=${OPENAI_API_KEY}
    - OPENAI_MODEL=${OPENAI_MODEL:-gpt-3.5-turbo}
    - REDIS_URL=redis://redis:6379
    - DID_VERIFY=${DID_VERIFY:false}
  ports: ["8022:8022"]
  depends_on:
    openhie-mediator:
      condition: service_started
    redis:
      condition: service_started

# ... (other agent services similarly defined)

command-centre:
  build:
    context: .
    dockerfile: shared/command-centre/Dockerfile
  environment:
    - REDIS_URL=redis://redis:6379
    - AGENT_URLS=http://triage-agent:8021,http://diagnosis-agent:8022, \
                 http://openhie-mediator:8023,http://transcriber-agent:8031, ...etc...
    - UPDATE_INTERVAL_MS=2000
  ports: ["8099:8099"]
  depends_on:
    - redis
    - triage-agent
    - diagnosis-agent
    - openhie-mediator
    # ... include all agents
```

(Comments: The actual file will list all 12 agent services with their respective env. We use `depends_on: service_started` rather than `healthy` for simplicity, except for mosquitto where we used a healthcheck.)

Deployment Workflow: To run the system, a developer will: 1. Set environment variables in `.env` (at least `NEXUS_JWT_SECRET` to a random string, and `OPENAI_API_KEY` for LLM agents to function, plus optionally `HITL_REQUIRED=false/true`, etc.). 2. Run `docker-compose up --build`. This will build all agent images (each Dockerfile will use a Python base image, install FastAPI, the nexus-common code, etc., then copy the agent code). Then it will start Redis, Mosquitto, HAPI FHIR, then each agent in dependency order, and finally the command-centre. 3. Verify all agents register as "healthy". Each agent's `/health` endpoint provides a JSON with status and basic metrics (the HealthMonitor collects task counts and latencies)[105][106]. The command centre will poll these and mark them green if OK. 4. The system is now ready for receiving tasks.

Integration with Nexus-A2A Protocol Code

We will heavily **reuse code from the `nexus-a2a-protocol` project** to speed up development and ensure compliance: - The **shared library** `nexus_common` from that repo provides modules for JSON-RPC handling, JWT auth, SSE events, HTTP client, and even DID verification. We will incorporate this either by copying the `shared/nexus_common/` directory into our project (as allowed, since this is an internal reference POC) or by installing the package if it's published. Reusing this means each agent's `main.py` can be very minimal – focusing only on the unique logic – while common features are one-liners to include. For example, adding authentication middleware and scope check is just using `_require_auth` as implemented[107], and parsing/dispatching RPC is done via `parse_request` and some boilerplate[1]. - We will follow the **same patterns** as the demo agents: - **FastAPI App Setup:** Create FastAPI instance, instantiate a `TaskEventBus` for SSE, a `HealthMonitor` for metrics, and use `logging.basicConfig` to configure logging similarly (so logs are uniform)[108][109]. - **Auth Middleware:** Define `_require_auth(request)` function in each agent that checks for the `Authorization: Bearer <token>` header and calls `verify_jwt(token, JWT_SECRET, required_scope)`[107]. This function will raise HTTP 401 if missing or invalid, which FastAPI will return as an error response. We apply this check on every RPC call and SSE connect (and WS connect) at the top of the endpoint handler[2][49]. This is exactly as in the reference code and satisfies the security requirements. - **Agent Card Endpoint:** Implement `@app.get("/.well-known/agent-card.json")` to return the static JSON (by reading the file)[56]. All agents do

this the same way, so we just ensure each has an up-to-date agent_card.json file (matching name, URL, methods, etc.). This allows a uniform discovery mechanism. - **Health Endpoint:** Implement `@app.get("/health")` returning `health_monitor.get_health()` as JSON[110]. This will include some metrics like tasks completed, error rates, etc. (the HealthMonitor in nexus_common already tracks `.metrics.record_*` calls around each task). - **Events Endpoint:** `@app.get("/events/{task_id}")` to stream SSE events. Use the `TaskEventBus.stream(task_id)` generator to yield events[2]. Also check auth on that request (the code shows `_require_auth` on SSE route)[2]. Similarly for WebSocket route: verify token query param and then accept and forward events[51]. - **RPC Endpoint:** `@app.post("/rpc")` to accept JSON-RPC calls. The pattern is to parse the request, dispatch to a local `METHODS` dict of handlers, and return either `response_result` or `response_error` accordingly[1][111]. We will implement each agent's specific methods and add them to the METHODS dict, then include the generic dispatcher code. The nexus-common `JsonRpcError` and helpers will be used to format errors properly. This yields a consistent JSON response structure across agents. - **Internal RPC calls:** When one agent calls another, we use the `shared.nexus_common.http_client.jsonrpc_call()` coroutine. This function handles making the HTTP request and parsing the response, including setting the `Authorization` header with a token we provide[48]. In the POC, since all agents trust the same secret, one agent can mint a JWT for itself to call others (some of the code minted a token for internal calls, e.g., Central uses `_make_token()` to call OSINT/Hospital over HTTP[112][113]). We will ensure that, e.g., TriageAgent passes along the same token it received when calling DiagnosisAgent (this was done in code by capturing `token = _require_auth()` then passing that token into `jsonrpc_call`[1][111]). This way the original caller's credentials propagate, and every agent along the chain sees a valid token. - **Error Handling:** By using the shared JSON-RPC utilities, we automatically comply with the expected error codes. For example, if a method throws an exception, we catch it and return a JSON-RPC error with code -32000 (Server error)[114]. If a method is not found, we return -32601 (Method not found)[115]. The scenario matrices have negative tests expecting these, so reusing this code ensures we pass those tests.

- **Agent-specific logic:** We will implement the custom logic inside each agent's handler function. But this is straightforward since we have the example code:
- TriageAgent's `_send_subscribe`: generate task_id, log, publish accepted, then asynchronously call DiagnosisAgent, publish final or error[11][116] – we'll mirror this logic. The agent card indicates TriageAgent supports `tasks/send` and `tasks/sendSubscribe` (and also `tasks/get`, `tasks/cancel` placeholders)[117]. For POC, we implement `tasks/sendSubscribe` fully. (We might not implement `tasks/send` if not needed – or make it just call sendSubscribe internally for compatibility.)
- DiagnosisAgent's `_assess`: as in code, fetch FHIR via openhie (with error handling to fallback to empty) and then do a simple reasoning to set priority and rationale[14][15]. We'll use the same LLM prompt approach provided.
- OpenHIEMediator's `_fhir_get`: perform HTTP GETs to HAPI for Patient and AllergyIntolerance and return JSON[19][20]. We use `httpx.AsyncClient` as in reference code with proper error catch[118].
- TranscriberAgent's `_send_subscribe`: similar to Triage, it publishes an accepted event, then sequentially calls Summariser and EHRWriter, publishing "working" status updates before each call[23][24]. We'll copy this flow. The final event will contain something like `{"saved": true, "ehr": {...}}` if all went well[22].
- SummariserAgent's `_summarise`: simply call `llm_chat(system_prompt, transcript)` and return the note text[25]. (We ensure to catch exceptions and return errors if the LLM call fails, to satisfy robustness).
- EHRWriterAgent's `_save`: open DB, insert record, return {"saved": true}[27]. And `_get_latest` if needed for testing (we may implement it since agent card lists it, and perhaps a scenario checks retrieval).
- InsurerAgent's `_send_subscribe`: publish accepted, call ProviderAgent (`records/provide`), wait for result[29], then if result says authorized and `HITL_REQUIRED=true`, call HITL agent for approval[30], attach that approval into the result, then publish final event[30][119]. We'll follow the reference closely.
- ProviderAgent's `_provide`: call ConsentAnalyser (`consent/check`), get `allowed` flag[33]. If not allowed, return authorized:false with reason[34]; if allowed, return authorized:true with a dummy record[120].
- ConsentAnalyserAgent's `_check`: call `llm_chat` with the consent text prompt, process output as allow/deny[35]. (Since this is quick, we may not need to stream any intermediate events here.)
- HITL-UI Agent's `_approve`: simply return an approval structure (we have it always approve in POC)[36].
- HospitalReporterAgent's `_surveillance_report`: generate random stats as done in the code snippet[37].
- OSINTAgent's `_osint_headlines`: lookup headlines in dict and return them[121].
- CentralSurveillanceAgent's `_tasks_send_subscribe` (or `surveillance/analyze`): implement the fan-out with `_try_mqtt_request` utility (given in code) to publish to `nexus/surveillance/report/req` topic and wait for a message on `.../resp` topic[41][42]. If that returns None (meaning no MQTT response), fallback to `jsonrpc_call` HospitalReporter's HTTP[42]. Do the same for OSINT on `nexus/osint/headlines/req/resp`[122]. Then call `llm_chat` to synthesize the alert JSON[44][45], parse it or wrap it, publish result event and return the compiled result[123].
- All these implementations are directly pulled from the expert POC artifacts (we have their logic from the repository), so coding them is mainly an exercise of adaptation. We will make sure to handle edge cases (e.g., timeouts, exceptions) similarly so tests expecting error events pass. For instance, OpenHIEMediator catches exceptions and returns empty context with a warning[124], ConsentAnalyser doesn't throw if LLM fails but returns denial by default, etc.

- **Logging and Monitoring:** We keep logging at INFO level for key events (as in code: logging an accepted and completed message with task and trace IDs[63][12]). These logs combined with the Command Centre's visual feed will help us debug. The `HealthMonitor` class from nexus_common will track how many tasks succeeded or errored, and average latency, which the command centre will display[105][106].

By reusing the Nexus A2A code and patterns, our implementation remains **consistent and interoperable** with the expected behavior. It also ensures that if we run the official **test harness** from the repository, our HelixCare agents will pass the same tests. For example, when the test for ED Triage calls `triage-agent` with a sample payload and then listens on `/events`, our agent will emit `nexus.task.status` and `nexus.task.final` events as expected[125]. If a test omits the Authorization header, our `_require_auth` will cause a 401, which the test expects as a pass for a negative scenario[126]. Thus, we maintain traceability: each requirement ID in the

matrix is satisfied by a combination of agent implementation and test verification (we can generate a conformance report to show coverage[127]).

Testing and Validation

To validate the full system, we will leverage the **scenario matrices** provided (traceability artifacts in JSON). Each JSON scenario describes a use-case step or test case with input and expected outcome. We will create a test harness that reads these scenarios and executes them against our running agents (very much like the `tests/nexus_harness` in the reference repo):

- We will include the scenario files for: `protocol_core`, `protocol_streaming`, `protocol_multitransport`, `ed_triage`, `telemed_scribe`, `consent_verification`, `public_health_surveillance` (the ones we have). These cover positive and negative tests for each major flow and requirement[4]. For instance, the **ED Triage matrix** has positive scenarios where a triage request is made and should succeed (covering FR-3, FR-4, FR-5 etc.) and negative ones (like missing auth) expecting failure[3][66].
- We can write a small `pytest` test module or an interactive script that for each scenario:
- If it's a discovery step (`protocol_step: agent_card_get`), perform an HTTP GET to the `.well-known/agent-card.json` of the target agent and check we got a 200 OK[57][58].
- If it's an RPC step (`"jsonrpc": "2.0"` in payload), determine which agent to call based on the method (the harness did this by checking substrings like "diagnosis" or "fhir" to route to the correct agent URL[128]). We can simplify by perhaps including the target agent in the scenario definition (if not, mimic their logic).
- Attach the Authorization header (the harness had a fixture `auth_headers` which likely provided a valid JWT with the correct scope). We can use our `NEXUS_JWT_SECRET` to pre-mint a token for tests, or in a live test use a known token that our agents accept (since they all share the secret, we could use a static token that `verify_jwt` will accept – e.g., generate one with the secret and `{"scope": "nexus:invoke"}`).
- Send the HTTP request to the agent's `/rpc` and capture the response. Verify the HTTP status matches `expected_http_status` in scenario (usually 200 for success, even for JSON-RPC errors since those come with 200 status by convention)[129]. Then check the JSON content:
    - If a `task_id` is returned (meaning an async call), and `expected_events` are listed, open the SSE stream at `/events/{task_id}` (with auth header) and listen for events[130]. Ensure that at least the events listed appear (the harness likely just waited until it got one of each type or timed out)[130]. We will verify that our agents indeed emit those events. For example, in a triage scenario, expected events might be `nexus.task.status` then `nexus.task.final` – our TriageAgent does publish a status ("accepted" or "working") and then final result, so that should match.
    - If it's a direct call (no async), then `expected_result` in the scenario can be compared to the `result` we got. For instance, a direct call to DiagnosisAgent's `diagnosis/assess` might have an expected_result containing `"triage_priority": "URGENT"` if the input chief complaint is non-critical. We can check important fields. However, because LLM output (rationale text) can vary, the harness might not strictly compare the exact string but just check that the call succeeded (`ok: true` in expected_result might just mean no error).
    - For negative scenarios, if `auth_mode` is "none", we omit the header and expect a failure. The harness marked pass if it got an error response in that case[131]. We'll replicate this logic – essentially ensuring unauthorized calls are indeed rejected.
- We will perform smoke tests manually for each main workflow:
- **ED Triage test:** Send a POST to TriageAgent's `/rpc` with a JSON like:

  ```
  {"jsonrpc": "2.0", "id": "1", "method": "tasks/sendSubscribe", "params": {"task": {"type": "ed-triage",
  "inputs": {"chief_complaint": "Chest pain"}}}}
  ```

  (with Authorization header). We expect immediate response containing a `task_id`. Then we GET `/events/{task_id}` and should receive events: one with `"state":"accepted"`, one with `"state":"working", "step":"calling_diagnosis"`, and finally one `nexus.task.final` with the result containing a triage priority (likely "EMERGENCY" due to "Chest pain") and a rationale text[16][15]. We also expect the DiagnosisAgent's side to possibly emit its own events if we listened (though triage collects and relays final result). We'll verify the final output has `triage_priority` and `patient_context` fields, confirming integration with FHIR data. This tests FR-3 (triage flow), FR-4 (AI diagnosis), FR-5 (FHIR lookup) all in one[3].
- **Telemed Scribe test:** POST to TranscriberAgent `tasks/sendSubscribe` with a transcript input (we can use a dummy conversation text). Expect `task_id` then events for "summarising" and "saving_ehr", then final. After completion, we could call EHRWriterAgent's `ehr/getLatestNote` to ensure the note was saved (or check its log/DB file). The final result from transcriber should indicate `{"saved": true}`[22]. This covers FR-7 (streaming updates) and overall documentation flow.
- **Consent Verification test:** POST to InsurerAgent `tasks/sendSubscribe` with a task containing `consent_text` (we'll input a sample consent text, perhaps "Patient consents to release of discharge summary to insurance."). We get a task_id and watch events. We expect: accepted, then "requesting_records", then maybe "provider_responded", then if HITL required, an event indicating waiting for approval (the POC sets state as `"input-required"` in that case)[30], then final. The final result should have either `authorized:false` if our consent text was not sufficient, or `authorized:true` with the `record` and a `hitl` sub-object (with approved:true) if it went through. We can test both scenarios by changing the consent text (e.g., include "not allowed" to trigger denial). This exercises SPR-1 (auth needed), and the consent use case FR-6 (agent discovery perhaps, though discovery is trivial here) and others.
- **Public Health test:** POST to CentralSurveillanceAgent `tasks/sendSubscribe` with params like `{"pathogen": "cholera", "region": "Gauteng"}`. We get a task_id. We should see an event for "working, step: fan-out"[47], then "working, step: synthesis", then final. The final result should include the combined alert with an alert_level and summary. We can verify that if MQTT is up, `transport_used` is "mqtt" in result[46]. If we bring down MQTT and try again, the agent should fall back to HTTP and set `transport_used: "http"` – this tests the multi-transport fallback logic (IR-2)[52][132].
- **Core protocol tests:** We also test a simple JSON-RPC call on each agent's basic method to ensure JSON-RPC compliance. For example, call a known method correctly (get a 200 result), call a nonexistent method and expect a proper JSON-RPC error

`{"error": {"code": -32601, "message": "Method not found"}}` (the harness likely does this). Also test error propagation: e.g., intentionally cause an error inside an agent (maybe by sending malformed params) to see it returns code -32000 server error. This ensures NFR-3 is met with correct error codes[72].

- **Security tests:** Attempt calls without auth or with bad tokens to ensure they are blocked (SPR-1). We can use an expired or wrong-signed JWT to test (the verify_jwt will throw AuthError and yield 401). The matrix's negative scenarios cover these and should pass[131].

We will automate these tests as much as possible. The target is to achieve a passing conformance run similar to the repository's own (noting that some tests might be skipped if certain features like DID are off). The traceability matrix provided maps each requirement to test cases and to our code modules[133]. After running the full suite, we can generate a **Conformance Report** JSON, listing each scenario and whether it passed, thus verifying that our implementation fulfills the intended requirements.

For example, we expect the report to show FR-3, FR-4, FR-5 covered by ED Triage demo scenarios all **pass**, FR-7 covered in Telemed and others pass, IR-2 covered in Public Health scenarios pass, NFR-1/2/3 and SPR-1 pass in all tests (these are tested across scenarios, e.g., every scenario enforces auth)[4]. If any fail, we'll debug and fix the logic accordingly – but since we are basing on working reference code, we anticipate a high pass rate.

Optional Frontend and Monitoring Tools

While the core of HelixCare is backend agents, a few frontend/monitoring components add value:

- **Command Centre Dashboard:** We have already included this as a service. Once the stack is running, a user can navigate to **http://localhost:8099** to view the live system status. This dashboard will display:
  - A network graph of all agents (nodes) and their RPC call relationships (edges)[134][135]. For instance, you'll see an arrow from TriageAgent to DiagnosisAgent (indicating triage calls diagnosis)[136], from CentralSurveillance to HospitalReporter/OSINT, etc. The nodes will be colored green/yellow/red based on their health (error rates, latency)[137]. Initially, all should be green. As tasks are executed, you'll see throughput numbers on the nodes and edges animating to show calls.
  - A real-time event timeline showing task events as they occur[138]. Each time an agent publishes an event (accepted, final, etc.), it will appear in the timeline with color coding (blue for accepted, violet for working, green for final, red for error, etc. per the legend)[139]. This is extremely useful to verify that, say, when we trigger a triage task, we see in the timeline: "triage-agent task accepted", then "diagnosis-agent subtask final", then "triage-agent task final" all with matching IDs. It basically provides a live log of the distributed workflow.
  - Performance charts (throughput, latency distribution, etc.) which are more relevant if we do load testing, but even with light load they show some metrics.
  - This command centre was implemented as a FastAPI app that polls `/health` and subscribes to Redis for events[140], so it should automatically pick up all our agents as long as AGENT_URLS is configured. We've listed all agent URLs in the compose config for it[99]. The agent health JSON includes tasks counts and last task time[141][142], which the UI uses.

Using the Command Centre, developers and stakeholders can visually confirm the system's behavior – for example, one can trigger each scenario and watch the events propagate in real time. This greatly aids in **demonstration** of the POC.

- **Human-in-the-Loop UI:** In a real deployment, the HITL-UI agent would be connected to an actual interface (web portal) for approvals. For the POC, since HITL is auto-approve, we do not have a user interface component to show. However, we could simulate it by having the command centre or another simple web page query the status of pending approvals. Since our HITL agent auto-responds immediately, we won't have a long-pending state. In a more advanced POC, we could implement the HITL agent to pause until a user clicks a button (perhaps via a REST endpoint to set the decision). If we wanted, we could expose the HITL agent's pending requests via its `/events` and allow a user to manually send the approve decision (for example, not auto-approve). But this adds complexity; for now we keep it automated and note this as future enhancement.

- **API Gateway or Clinician UI:** Outside the core scope, it's worth noting how actual users enter the loop:
  - For ED Triage, a nurse could have a UI that sends the initial triage request to the TriageAgent (e.g., via a web form or mobile app). In our test, we simulate that via HTTP calls. A simple web form could be built to take patient info and call the agent's API, then display the result. For demonstration, we might prepare a small script or use tools like `curl`/Postman to emulate the client.
  - For Telemed Scribe, ideally the system would listen to an audio stream, transcribe it (via a speech-to-text service, which could be another agent), then feed text to TranscriberAgent. We are skipping the audio part in POC. If needed, we could stub an "audio transcriber agent" that simply takes an input text (pretending it did speech recognition). But that might be overkill; we can assume the transcript is given.
  - These potential UI integrations are outside the immediate ask, but our architecture supports adding them (they'd just become additional agents or clients interacting via the same Nexus protocol or via a dedicated API gateway that translates external requests into Nexus tasks).

- **Scaling & Next Steps:** Our implementation plan is focused on a single-hospital POC. In the future, one might separate these agents onto different nodes, secure the network (which mTLS covers), and include more agents (e.g., a SchedulingAgent, a BillingAgent for revenue cycle, a DischargeAgent as mentioned in the white paper's expansion use cases[143][144]). The modular design and directory structure allows adding new agents easily.

- **Compliance and Auditing:** Because healthcare is regulated, we note that our architecture inherently supports auditing: all events can be recorded (via command centre or a logging pipeline) to provide a detailed trace of who did what (which agent, acting under which token, did which task for which patient). We have requirement mappings and conformance tests to ensure nothing was missed. If needed, we could output an **audit log** from each agent when a final task completes (e.g., log patient identifiers or task details). Given ISO 29148 focus on requirements, our scenario matrices and conformance report serve as proof that requirements (functional and security) were met in the POC.

Finally, we document how to run some example **smoke test** manually, so the team can quickly verify the POC: - *Example:* **ED Triage Smoke Test** – 1. Send a triage request:

```
curl -X POST http://localhost:8021/rpc \
     -H "Authorization: Bearer $TEST_TOKEN" \
     -d '{"jsonrpc":"2.0","id":"req1","method":"tasks/sendSubscribe",
          "params":{"task":{"type":"ed-triage","inputs":{"chief_complaint":"Chest pain"}}}}'
```

(Replace $TEST_TOKEN with a JWT signed by our secret, containing `"scope":"nexus:invoke"`). 2. The response will be `{"jsonrpc":"2.0","id":"req1","result":{"task_id":"...","trace_id":"..."}}`. Extract the task_id. 3. Connect to SSE:

```
curl -N -H "Authorization: Bearer $TEST_TOKEN" http://localhost:8021/events/<task_id>
```

You should see events like:

```
data: {"task_id":"...","state":"accepted","trace_id":"..."}
data: {"task_id":"...","state":"working","step":"calling_diagnosis"}
data: {"triage_priority":"EMERGENCY","rationale":"Patient likely having MI...","patient_context":{...}}
```

The last event (nexus.task.final) contains the result JSON. This confirms the workflow. - We would provide similar instructions for other flows or simply instruct to run `pytest tests/` to execute the automated scenarios.

By following this comprehensive implementation plan, we will build the HelixCare AI Hospital POC as a fully working system. It will utilize all artifacts from the expert design (we've essentially transcribed those designs into code and config), and it will demonstrate how an autonomous network of agents can perform complex clinical and operational tasks securely. Each part of the plan was derived from the provided specifications and scenario matrices, ensuring that the final deliverable is traceable to the stated requirements and can serve as a blueprint for a real-world autonomous hospital system.

---

[1] [2] [11] [12] [48] [50] [51] [56] [60] [63] [64] [69] [70] [71] [73] [107] [108] [109] [110] [111] [116] main.py

https://github.com/sync-ai-health/nexus-a2a-protocol/blob/30a9379813d0af8e4d06c246a9370b35688dedef/demos/ed-triage/triage-agent/app/main.py

[3] [4] [52] [53] [61] [62] [65] [66] [72] [127] [132] [133] traceability-matrix.md

https://github.com/sync-ai-health/nexus-a2a-protocol/blob/30a9379813d0af8e4d06c246a9370b35688dedef/docs/traceability-matrix.md

[5] [6] [9] [59] [74] [143] [144] autonomous_digital_hospital_white_paper.md

https://github.com/sync-ai-health/nexus-a2a-protocol/blob/30a9379813d0af8e4d06c246a9370b35688dedef/docs/autonomous_digital_hospital_white_paper.md

[7] [8] [10] [117] agent_card.json

https://github.com/sync-ai-health/nexus-a2a-protocol/blob/30a9379813d0af8e4d06c246a9370b35688dedef/demos/ed-triage/triage-agent/app/agent_card.json

[13] [67] [68] agent_card.json

https://github.com/sync-ai-health/nexus-a2a-protocol/blob/30a9379813d0af8e4d06c246a9370b35688dedef/demos/ed-triage/diagnosis-agent/app/agent_card.json

[14] [15] [16] [49] [114] [115] main.py

https://github.com/sync-ai-health/nexus-a2a-protocol/blob/30a9379813d0af8e4d06c246a9370b35688dedef/demos/ed-triage/diagnosis-agent/app/main.py

[17] [18] agent_card.json

https://github.com/sync-ai-health/nexus-a2a-protocol/blob/30a9379813d0af8e4d06c246a9370b35688dedef/demos/ed-triage/openhie-mediator/app/agent_card.json

[19] [20] [118] [124] main.py

https://github.com/sync-ai-health/nexus-a2a-protocol/blob/30a9379813d0af8e4d06c246a9370b35688dedef/demos/ed-triage/openhie-mediator/app/main.py

[21] [22] [23] [24] main.py

https://github.com/sync-ai-health/nexus-a2a-protocol/blob/30a9379813d0af8e4d06c246a9370b35688dedef/demos/telemed-scribe/transcriber-agent/app/main.py

[25] main.py

https://github.com/sync-ai-health/nexus-a2a-protocol/blob/30a9379813d0af8e4d06c246a9370b35688dedef/demos/telemed-scribe/summariser-agent/app/main.py

[26] agent_card.json

https://github.com/sync-ai-health/nexus-a2a-protocol/blob/30a9379813d0af8e4d06c246a9370b35688dedef/demos/telemed-scribe/summariser-agent/app/agent_card.json

[27] [28] main.py

https://github.com/sync-ai-health/nexus-a2a-protocol/blob/30a9379813d0af8e4d06c246a9370b35688dedef/demos/telemed-scribe/ehr-writer-agent/app/main.py

[29] [30] [119] main.py

https://github.com/sync-ai-health/nexus-a2a-protocol/blob/30a9379813d0af8e4d06c246a9370b35688dedef/demos/consent-verification/insurer-agent/app/main.py

[31] [32] [33] [34] [120] main.py

https://github.com/sync-ai-health/nexus-a2a-protocol/blob/30a9379813d0af8e4d06c246a9370b35688dedef/demos/consent-verification/provider-agent/app/main.py

[35] main.py

https://github.com/sync-ai-health/nexus-a2a-protocol/blob/30a9379813d0af8e4d06c246a9370b35688dedef/demos/consent-verification/consent-analyser/app/main.py

[36] main.py

https://github.com/sync-ai-health/nexus-a2a-protocol/blob/30a9379813d0af8e4d06c246a9370b35688dedef/demos/consent-verification/hitl-ui/app/main.py

[37] [38] main.py

https://github.com/sync-ai-health/nexus-a2a-protocol/blob/30a9379813d0af8e4d06c246a9370b35688dedef/demos/public-health-surveillance/hospital-reporter/app/main.py

[39] [40] [121] main.py

https://github.com/sync-ai-health/nexus-a2a-protocol/blob/30a9379813d0af8e4d06c246a9370b35688dedef/demos/public-health-surveillance/osint-agent/app/main.py

[41] [42] [43] [44] [45] [46] [47] [112] [113] [122] [123] main.py

https://github.com/sync-ai-health/nexus-a2a-protocol/blob/30a9379813d0af8e4d06c246a9370b35688dedef/demos/public-health-surveillance/central-surveillance/app/main.py

[54] [87] [88] [93] docker-compose.yml

https://github.com/sync-ai-health/nexus-a2a-protocol/blob/30a9379813d0af8e4d06c246a9370b35688dedef/demos/telemed-scribe/docker-compose.yml

[55] [80] [81] [82] [83] [84] [85] [86] [92] docker-compose.yml

https://github.com/sync-ai-health/nexus-a2a-protocol/blob/30a9379813d0af8e4d06c246a9370b35688dedef/demos/ed-triage/docker-compose.yml

[57] [58] [125] [126] [128] [129] [130] [131] test_ed_triage.py

https://github.com/sync-ai-health/nexus-a2a-protocol/blob/30a9379813d0af8e4d06c246a9370b35688dedef/tests/nexus_harness/test_ed_triage.py

[75] [76] [99] [101] [105] [106] [138] [140] command-centre-summary.md

https://github.com/sync-ai-health/nexus-a2a-protocol/blob/30a9379813d0af8e4d06c246a9370b35688dedef/docs/command-centre-summary.md

[77] [78] [79] [96] docker-compose.yml

https://github.com/sync-ai-health/nexus-a2a-protocol/blob/30a9379813d0af8e4d06c246a9370b35688dedef/demos/public-health-surveillance/docker-compose.yml

[89] [90] [91] [94] [95] [97] [98] [102] docker-compose.yml

https://github.com/sync-ai-health/nexus-a2a-protocol/blob/30a9379813d0af8e4d06c246a9370b35688dedef/demos/consent-verification/docker-compose.yml

[100] [103] [104] [134] [135] [136] [137] [139] [141] [142] README.md

https://github.com/sync-ai-health/nexus-a2a-protocol/blob/30a9379813d0af8e4d06c246a9370b35688dedef/shared/command-centre/README.md